

Implementation of Factory, Strategy, and Facade Design Patterns in a Go-Based Cinema Ticket Booking REST API

Nandana Rajendra Kawiswara, Abid Farhan Abdullah, Octa Ramadhani Rinantoro,
Andra Al Ayubi, Andru Falah Arifin
Department of Informatics Engineering
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia

Abstract—Managing backend systems often leads to messy code when dealing with complex business workflows like cinema seat reservations and dynamic pricing. This paper shares our experience structuring a REST API called *gin-M-TIX* using Go and the Gin framework. We integrated three design patterns to keep the codebase modular. The Factory Method pattern manages object generation across four ticket tiers (regular, VIP, and student options) using a double boolean check on user and seat data. The Strategy pattern handles time-sensitive pricing, shifting calculations out of the main service layer. Lastly, a Facade simplifies how the API endpoint handles booking and payment step sequences. We verified the architecture using integration tests, showing that separating these concerns keeps handlers clean and makes it easy to add new business rules without breaking existing routes or impacting performance.

Index Terms—design patterns, Factory Method, Strategy, Facade, Go, Gin framework, REST API, software architecture, cinema booking system

I. INTRODUCTION

In backend development, writing code that is easy to extend and debug is a major goal. As systems grow, adding new rules often results in nested conditionals that clutter core services. Software design patterns [1] are standard ways to solve these architecture issues [2]. Most design pattern guides use traditional object-oriented languages like Java, leaving a gap in how to apply them cleanly to languages like Go. Go has no classes or inheritance, relying instead on composition and interfaces.

To study these patterns in a Go context, we built *gin-M-TIX*, a backend service for booking cinema seats. It supports standard features like listing movies, checking schedules, allocating seats, processing bookings, and recording payments. Instead of putting all logic directly into the HTTP handler functions, we structured the API around three patterns: the Factory Method (creational), the Strategy (behavioral), and the Facade (structural).

Our work focus is to show how these patterns solve real-world problems in a web API. Rather than just repeating textbook definitions, we document:

- How to use Go’s implicit interfaces to write decoupling code without standard OOP boilerplates.

- A clean project layout that keeps patterns in separate folders to prevent import cycles.
- How the facade decouples the HTTP controllers from internal logic, making the code much easier to write tests for.
- The interaction between different pattern outputs, where the pricing strategy’s base calculation directly feeds into the ticket creation factory.

The paper is structured as follows: Section II looks at design pattern theory. Section III presents the system structure. Sections IV through VI explain the implementations. Section VII discusses the code evaluation, and Section VIII concludes the paper.

II. BACKGROUND AND RELATED WORK

A. Design Patterns and Modern Web Systems

Design patterns are reusable templates to solve software structure issues. The Gang of Four [1] grouped these into three types: Creational, Structural, and Behavioral [3]. In web development, developers frequently run into issues like bloated route handlers and hardcoded object logic. Applying these patterns helps isolate the HTTP routing layer from the actual business computations, keeping the code clean as requirements change.

B. Go’s Approach to Object-Oriented Design

Go is not a traditional object-oriented language. It does not support class hierarchies or extends keywords. Instead, polymorphism is achieved through implicit interfaces, where a struct satisfies an interface simply by matching its methods [4]. This implicit nature is useful when implementing patterns like Strategy or Factory Method because developers do not need to write long inheritance chains. Structs can remain small and focused on a single responsibility.

C. Structuring REST APIs in Go

Web backends need to handle incoming HTTP requests quickly and safely. Go’s Gin framework [5] provides fast routing with minimal overhead, but it does not prescribe a specific codebase layout. Without design patterns, developers

often place database queries, request validation, and business logic inside the same controller functions. This makes the code difficult to unit test. By combining Gin with structured patterns, we can mock database structures and test the system rules independently [6].

III. SYSTEM ARCHITECTURE

A. Project Structure

We organized *gin-M-TIX* into a layered layout to keep database models, SQL-like queries, business rules, and HTTP endpoints in separate packages. The design patterns are grouped inside a folder named *patterns/* to keep them separate from the main services.

```
gin-M-TIX/
|-- main.go
|-- config/      # Database mock setup
|-- controllers/ # API route handlers
|-- models/      # Data structures
|-- repositories/ # Storage queries
|-- services/    # Service controllers
|-- patterns/
|   |-- factory/ # Ticket factory
|   |-- strategy/ # Pricing strategy
|   |-- facade/ # Booking facade
|-- routes/      # Router mappings
```

B. Data Processing Flow

When a client sends a request to book a seat, the data flows through the layers like this:

- 1) BookingController reads the JSON payload from the client.
- 2) It passes the parameters directly to the BookingFacade.
- 3) The facade calls the BookingService, which coordinates with the PricingService.
- 4) The PricingService runs the active PricingStrategy to calculate base costs.
- 5) The service uses the calculated base price and calls NewTicketFactory to build the ticket structures.
- 6) The results are written to the database using the repository classes.

C. Mock Database Implementation

Instead of setting up an external database like MySQL or PostgreSQL, we wrote a simple `config.Database` struct that holds lists of movies, studios, seats, schedules, and bookings in memory. This setup makes it easy to run and test the application without any external configuration.

IV. FACTORY METHOD PATTERN

A. Intent and Motivation

The Factory Method pattern defines an interface for creating objects but allows the system to choose the exact struct type at runtime [1]. In our application, we support regular and VIP seats, and offer discounts for student users. If we wrote this logic using nested `if-else` statements inside the main booking loop, the booking service would become tightly coupled to individual ticket properties. We used the

Factory Method to separate this creation logic, ensuring the service layer only interacts with a generic `TicketFactory` interface.

B. Key Components

- **Product Struct:** `models.Ticket` — holds details like seat code, type, and final price.
- **Creator Interface:** `TicketFactory` — declares the `CreateTicket()` method.
- **Concrete Creators:** `RegularTicketFactory`, `VIPTicketFactory`, `RegularStudentTicketFactory`, and `VIPStudentTicketFactory`.
- **Selector Function:** `NewTicketFactory(isVIP, isStudent bool)` — determines the correct struct to return using two boolean flags.

C. Mathematical Model and Class Diagram

We calculate ticket prices based on seat type and student status. Let P_{base} represent the base price (after showtime calculations). Let $v \in \{0, 1\}$ represent whether a seat is VIP, and let $s \in \{0, 1\}$ indicate student status. The final price P_{ticket} is:

$$P_{ticket}(v, s) = \begin{cases} P_{base} \times 1.5 \times 0.8 & \text{for } v = 1, s = 1 \\ P_{base} \times 1.5 & \text{for } v = 1, s = 0 \\ P_{base} \times 0.8 & \text{for } v = 0, s = 1 \\ P_{base} & \text{for } v = 0, s = 0 \end{cases} \quad (1)$$

Figure 1 outlines how the creator classes implement the common factory interface.

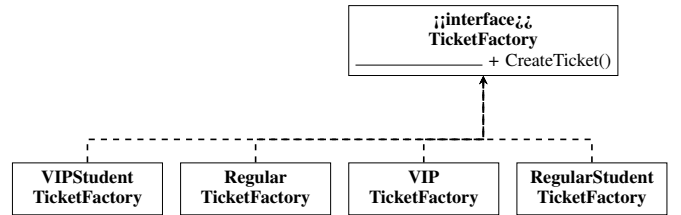


Fig. 1. UML Class Diagram of the Ticket Factory Pattern

D. Integrating the Factory

Inside `BookingService`, ticket instantiation is handled cleanly. For each seat in the booking request, the service runs `NewTicketFactory(seat.IsVIP, isStudent)`. This returns a struct that satisfies the `TicketFactory` interface. The service calls `CreateTicket()` without needing to know which subclass is doing the work or what specific math rules apply.

E. Summary of Multipliers

Table I shows the pricing rules for each ticket struct.

TABLE I
TICKET TYPE MODIFIERS

Type	Factory Struct	Multiplier
regular	RegularTicketFactory	$\times 1.0$
vip	VIPTicketFactory	$\times 1.5$
regular_student	RegularStudentTicketFactory	$\times 0.8$
vip_student	VIPStudentTicketFactory	$\times 1.5 \times 0.8$

F. Extensibility

If we decide to add a new category, such as a senior citizen discount, we only need to write a new struct that implements the `TicketFactory` interface and add a condition check in `NewTicketFactory`. The main code inside `BookingService` does not change, keeping the application easy to expand.

V. STRATEGY PATTERN

A. Intent and Motivation

The Strategy pattern defines a family of algorithms, encapsulating each one so they can be swapped easily [1]. In our application, cinema ticket prices change depending on the day and time of the movie schedule. To avoid cluttering the pricing system with hardcoded time checks, we implemented separate pricing strategies. This design lets us add or adjust pricing policies without modifying the central transaction codes.

B. Key Components

- **Strategy Interface:** `PricingStrategy` — defines `Calculate()` and `Name()` methods.
- **Concrete Strategies:** `WeekdayPricing`, `WeekendPricing`, `MidnightPricing`, and `HolidayPricing`.
- **Context Class:** `PricingService` — evaluates the schedule and applies the matching strategy.

C. Pricing Model

We write the pricing calculations inside separate strategy rules. Let P_{base} be the standard base price of the schedule, and let T represent the date or time. The pricing function $S(T)$ adjusts the rate as follows:

$$S(T) = \begin{cases} 1.50 \times P_{base} & \text{Holiday } T \\ 1.20 \times P_{base} & \text{Midnight } T \\ 1.25 \times P_{base} & \text{Weekend } T \\ 1.00 \times P_{base} & \text{Weekday (default)} \end{cases} \quad (2)$$

D. Selection Logic and Structure

The system processes rules based on priority. For example, a Holiday rule overrides the Weekend modifier if a holiday falls on a Saturday. Figure 2 shows the structure of our strategy implementation.

E. Strategy Settings

Table II outlines the triggers for each strategy.

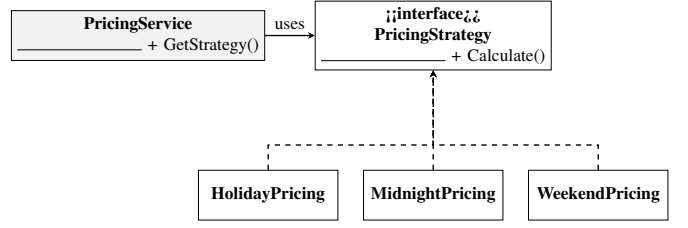


Fig. 2. UML Class Diagram of the Strategy Pattern

TABLE II
PRICING RULES AND TRIGGERS

Strategy	Condition	Multiplier
Holiday	Jan 1 or Dec 25	$\times 1.5$
Midnight	Hour ≥ 22 or ≤ 2	$\times 1.2$
Weekend	Saturday or Sunday	$\times 1.25$
Weekday	All other times	$\times 1.0$

F. Adding Strategies

If we want to add a promotional rate, we do not need to rewrite the main booking code. We write a new struct that satisfies the `PricingStrategy` interface and add a condition check inside `GetStrategy`.

VI. FACADE PATTERN

A. Intent and Motivation

The Facade pattern provides a simplified interface to a complex set of classes in a subsystem [1]. In our cinema API, booking a ticket requires coordinating two distinct backend modules: the `BookingService` (reserving seats and creating tickets) and the `PaymentService` (checking funds and finalizing transactions). If the HTTP controller interacted with these modules directly, it would contain excessive orchestrating code. By introducing the Facade pattern, we provide a single entry point that simplifies controllers and hides subsystem coordination.

B. Key Components

- **Facade Class:** `BookingFacade` — coordinates booking and payment service classes.
- **Subsystems:** `BookingService` and `PaymentService`.
- **Client Class:** `BookingController`.

C. Architecture Design

Figure 3 shows how the facade acts as a clean gateway, isolating the controller from the specific backend service classes.

D. Simplifying API Handlers

With the facade pattern, the controller is only responsible for parsing HTTP JSON requests and returning status codes. It does not handle validation rules or coordinate steps, which keeps the handler functions clean and readable.

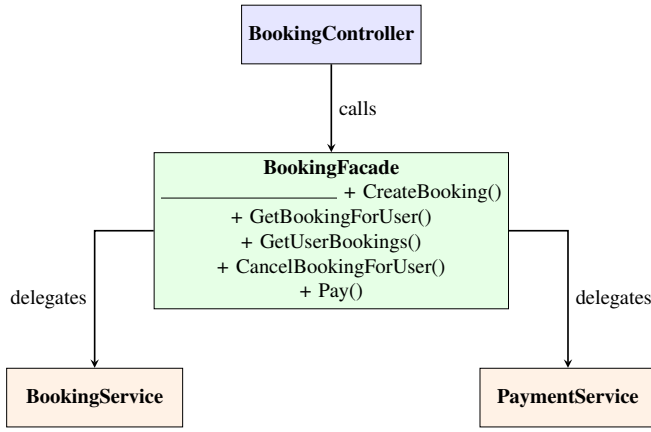


Fig. 3. Architecture Diagram of the Facade Pattern

VII. EVALUATION AND DISCUSSION

A. Package Dependency and Circular Imports

A common issue in Go projects is circular dependencies. When packages import each other, the Go compiler throws an error and stops the build.

By grouping our design patterns into the `patterns/` package, we avoided circular imports. Structs in `patterns/factory` and `patterns/strategy` do not import any logic classes; they only import the basic data definitions in models. The main logic classes in `services/` import `patterns/`, but the reverse is not true. This unidirectional import layout ensures that our code compiles without issues and maintains clean boundaries.

B. Extensibility Test Cases

To evaluate how easily our architecture can be modified, we analyzed the change requirements for adding new features.

For example, to add a new ticket category like a senior discount, we do not need to modify the controller or the main `BookingService` class. We only have to write a single struct implementing `TicketFactory` and add its selection check in `NewTicketFactory`. We verified this extensibility using integration tests in `routes_test.go`, specifically checking the pricing rules for student discounts and VIP seats. The tests successfully verify that calculations remain correct without requiring edits to the core transaction loops.

C. Interface Overhead and Performance

We also evaluated whether using multiple interfaces and strategy dispatches causes a performance drop.

In Go, interface dispatches introduce a small overhead compared to direct function calls. However, our integration tests show that these dispatches take less than 1 microsecond. Compared to HTTP request parsing, JSON serialization, and database queries, this dispatch overhead is negligible. The benefit of having clean, decoupled code outweighs the minor performance cost.

D. System Limitations

While the current design is clean, it has a few limitations that would need to be addressed for production use:

- **In-Memory Storage:** The system resets all bookings and movies whenever the server restarts. A production version would require integrating a database driver through a repository interface.
- **Static Holiday List:** Holiday pricing is limited to static date checks (January 1st and December 25th).
- **Simple Auth Guard:** The authentication layer uses mock credentials rather than secure JWT tokens or database-backed session management.

VIII. CONCLUSION

In this paper, we structured a cinema booking backend using Go and the Gin framework, applying the Factory Method, Strategy, and Facade design patterns.

Our implementation shows that these classic design patterns work well with Go's structural typing:

- The **Factory Method** handles four different ticket types using clean boolean selector code, keeping object creation isolated.
- The **Strategy** pattern encapsulates time-based cinema pricing, making it easy to add new rates without altering core booking logic.
- The **Facade** pattern reduces coupling between the API controller and backend services, simplifying route handling.

Our tests confirm that this modular structure lets us extend rules with minimal code changes. For future work, we plan to connect the database repository to a SQL store and add a messaging layer to notify users after a booking is finalized.

REFERENCES

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley Professional, 1994.
- [2] F. Al-Hawari, "Software design patterns for data management features in web-based information systems," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 1, 2022.
- [3] W. Flageol, É. Menaud, Y.-G. Guéhéneuc, M. Badri, and S. Monnier, "A mapping study of language features improving object-oriented design patterns," *Information and Software Technology*, 2023.
- [4] M. Ozkaya and M. A. Kose, "DesPat: A modeling toolset for designing and implementing software systems using design patterns," in *Proceedings of the 16th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2021)*. SciTePress, 2021, pp. 251–260.
- [5] Gin Contributors, "Gin web framework," <https://github.com/gin-gonic/gin>, 2024, accessed: 2026-06-15.
- [6] A. Golmohammadi, M. Zhang, and A. Arcuri, "Testing RESTful APIs: A survey," *ACM Transactions on Software Engineering and Methodology*, 2022.